

libirsample 程序结构以及使用流程

目录

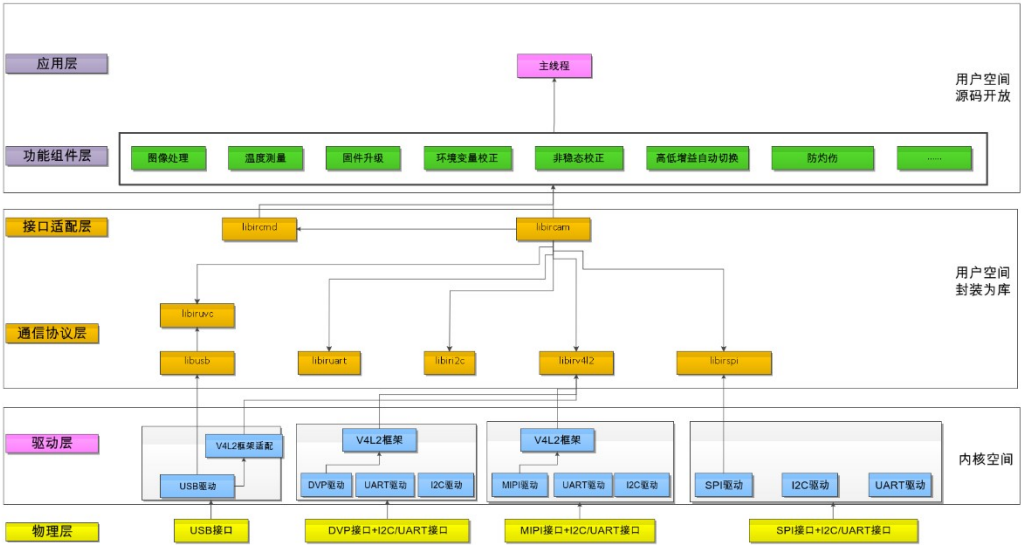
简介.....	2
结构说明.....	3
基本功能组件 commom.....	3
特殊功能组件 components.....	3
驱动层 drivers.....	3
接口层 interfaces.....	3
VS 工程文件 msvc.....	4
其他 SDK 接口 other.....	4
应用层 sample.....	4
第三方依赖库 thirdparty.....	4
程序编译方式.....	5
windows 平台.....	5
linux 平台.....	5
基础组件说明.....	6
1.连接机芯.....	6
2.控制出图.....	6
3.发送命令.....	7
4.结束程序.....	8
特殊组件说明.....	9
1.升级功能.....	9
2.测温二次标定.....	12
3.盲元标定.....	13
4.测温环境变量修正功能.....	13
5.点线框测温说明.....	14
6.高精度时钟说明.....	14
不同模式数据流.....	16

简介

libir_sample 包含了用于展示机芯多种功能的示例，例如获取数据流、图像输出显示、命令发送、信息行解析、固件升级、环境变量修正、温度测量、截图和录像等功能。

整个 SDK 系统主要分为了应用层、功能组件层、接口适配层、通信协议层、驱动层和物理层等。其中应用层、功能组件层在 libir_sample 中进行实现，开放源码。接口适配层、通信协议层在用户空间，且封装为库，仅对客户开放接口。驱动层在内核空间。

从下层到上层，基本上是上层依赖下层的接口。各个层级只负责对应层的功能，不干涉其它层级。设计框架如下：



结构说明

基本功能组件 **common**

包括绝大部分用户都需要用到的功能比如获取数据流、图像输出显示、数据变量初始化等功能将放到 common 目录。

camera 模块：用于获取机芯信息，当 stream_thread 线程获取到原始红外帧信息的时候，会将红外帧信息 raw frame 切分为图像信息 image frame 和温度信息 temp frame，并发送信号，对 image frame 进行处理，当 image frame 处理完成后发送信号给 camera 线程，camera 线程继续下一次循环。

data 模块：定义、初始化、释放一些全局的数据变量。

opencv_display 模块：获取图像帧信息之后，根据不同的图像数据类型做图像数据格式转换等处理。

drm_display 模块：调用 linux 环境的第三方依赖库 libdrm，进行图像的显示。

transform 模块：多种数据格式转换的示例。

注意，当前的显示是 RV1126 开发板上的配置，且分辨率为 640*512，若需要更换其他开发板修要修改配置。

特殊功能组件 **components**

部分客户需要用到的功能组件，例如命令发送、信息行解析、固件升级、环境变量修正、温度测量、截图和录像等相关功能。

capture 模块：实现截图及录像功能。

cmd 模块：调用依赖库 libircmd 的接口，给红外机芯发送一些简单的控制命令。

info_parse 模块：解析数据帧中的图像信息行内容，并将所有信息按照状态、功能、测试进行分类，调用不用的接口即可拿到不同类型的数据。

overexposure 模块：实现防过曝功能。

temp_correct 模块：实现温度修正的功能。

temp_measure 模块：通过使用依赖库 libircmd 的接口、info_parse 模块的信息行数据、或依赖库 libirtemp 中对温度帧的解析，实现获取点线框的温度功能。

upgrade 模块：使用更新包做不同版本的完整包（固件+数据）之间的更新，更新包由软件 UpdatePacker 产生。

驱动层 **drivers**

sample 或者组件需要依赖的和硬件相关的库例如 libirv4l2、libiruvic、libiri2c、libiruart、libirspi 等。

libiri2c 模块：用于通过 I2C 接口和机芯进行通信，封装了自定义的指令协议。

libirspi 模块：用于通过 SPI 接口和机芯进行通信，封装了自定义的指令协议。

libiruart 模块：用于通过 UART 接口和机芯进行通信，封装了自定义的指令协议

libiruvic 模块：对使用 USB 及 I2C_USB 的传输方式进行封装。

libirv4l2 模块：对使用 USB、DVP+I2C/UART、MIPI+I2C/UART 的传输方式进行封装。

接口层 **interfaces**

sample 或者组件需要依赖的抽象接口层，如 libircamera、libircmd 等。

libircam 模块：使用依赖倒置的方式，被应用层和驱动层共同依赖，从而实现应用层与机芯数据交互时无需考虑驱动类别。该模块的 handle 由底层驱动注册，由应用层直接调用。

libircmd 模块：进行与机芯的数据交互。该模块将 vdcmd 命令格式转换成对应通信协议层可以接收的数据格式，并提供给用户可以直接调用的函数。

libirdfu 模块：提供固件更新的接口。

pack_tool 模块：进行一些数据更新读写压缩包的操作，被特殊功能组件层的 upgrade 模块调用。提供直接从压缩包中读写文件的接口，以及向 JSON 文件中添加键值对的接口。

VS 工程文件 msvc

在 windows 上启动程序需要配置的 vs 工程文件，在 windows 平台，打开 libirsample.sln 即可打开整个工程。

其他 SDK 接口 other

sample 或者组件需要依赖的一些英菲内部使用的依赖库。这些库文件均是软件层面的数据处理函数库，客户可根据具体的需求选用相关的函数接口。

libirprocess 模块：实现了图像的旋转、镜像、翻转、图像增强、空域滤波等功能

libirparse 模块：实现了图像数据格式转换功能

libirtemp 模块：实现了使用解析温度帧的方式进行点、线、框测温，并提供了相关的测温修正函数

应用层 sample

作为应用层，使用上方提供的基础功能组件层及特殊功能组件层，实现能达到不同功能的示例，用于给用户作为演示及参考。

Info_line_parse 示例：针对图像带信息行的机芯进行信息行信息的解析。当前只适配 linux 平台。

mipi_2vc_stream_cmd 示例：针对 MIPI 接口双 VC 输出的数据流接收和串口发送命令使用示例。当前已经在 RV1126 平台上进行了验证。

uart_cmd 示例：使用串口进行发送指令的示例。

upgrade 示例：多种接口固件更新和数据升级的示例。

uart_upgrade 示例：使用串口进行固件更新的示例。

stream_cmd 示例：针对 MIPI 接单 VC 输出的数据流接收和串口发送命令使用示例。当前已经在 RV1126 平台上进行了验证。

windows_usb_stream_cmd 模块：可实现获取数据流、图像输出显示、发送基础控制命令的示例。

第三方依赖库 thirdparty

sample 或者组件需要依赖的第三方库例如 libdrm、opencv、ffmpeg 等库。当前 linux 平台显示依赖 libdrm 库，windows 平台显示依赖 opencv 库。

程序编译方式

windows 平台

msvc 文件夹下已经提供了一个完整的 VS 工程，运行示例需要 opencv 库和 pthreadVC2.dll。

linux 平台

在 linux 平台，针对不同的功能示例，在对应的功能示例文件夹下，提供了 CMakeLists.txt 文件。在编译时，图像显示使用的是 libdrm，如果不需要 libdrm，可在 CMakeLists.txt 中注释掉 libdrm 相关内容，然后再编译。

基础组件说明

1.连接机芯

在获取到参数信息之后，还需要调用 load_stream_frame_info 函数来补充对 display 和 temperature 两个模块的设置，比如宽高、数据格式等信息，申请 buffer 空间等。

如下是 StreamFrameInfo_t 结构体的定义，注意这几项参数：

FrameInfo_t-data_height/info_height/data_info_height:数据帧的数据、信息行以及总和高度，需要填写

FrameInfo_t-input_format/output_format:输入和输出的数据格式，比如 Y14 数据输入，RGB888 输出，需要填写

StreamFrameInfo_t-image_byte_size/temp_byte_size:输入帧数据的字节大小，填 0 即收不到数据，根据需要切分的数据大小来填写

```
typedef struct {
    uint32_t data_height;
    uint32_t info_height;
    uint32_t data_info_height;
    InputFormat_t input_format;
    OutputFormat_t output_format;
}FrameInfo_t;

typedef struct {
    char *name;
    void* driver_handle;
    IrcmdHandle_t* ircmd_handle;
    uint32_t width;
    uint32_t height;
    uint8_t* raw_frame;
    uint32_t raw_byte_size;
    uint8_t* image_frame;
    uint32_t image_byte_size;
    uint8_t* temp_frame;
    uint32_t temp_byte_size;
    FrameOutputFormat_t frame_output_format;
    FrameInfo_t image_info;
    FrameInfo_t temp_info;
}StreamFrameInfo_t;
```

2.控制出图

在打开设备，获取到相关的参数信息，并补充完对 display 模块的参数设置之后，就可以调用 stream_function 来获取数据帧了。

```
while (is_streaming && (i <= STREAM_TIME * stream_frame_info->camera_param.fps))//display stream_time seconds
{
```

```

#if defined(_WIN32)
    WaitForSingleObject(image_done_sem, INFINITE);
#elif defined(linux) || defined(unix)
    sem_wait(&image_done_sem);
#endif

    r = iruvvc_frame_get(stream_frame_info->iruvvc_handle, stream_frame_info->raw_frame);
    if (r < 0)
    {
        overtime_cnt++;
    }
    else
    {
        overtime_cnt = 0;
    }
    while (r < 0 && overtime_cnt >= overtime_threshold)
    {
#if defined(_WIN32)
        ReleaseSemaphore(image_sem, 1, NULL);
#elif defined(linux) || defined(unix)
        sem_post(&image_sem);
#endif

        ir_camera_stream_off(stream_frame_info);
        printf("uvcc_frame_get failed\n ");
        return NULL;
    }
    if (stream_frame_info->raw_frame != NULL)
    {
        raw_data_cut((uint8_t*)stream_frame_info->raw_frame, stream_frame_info->image_byte_size, \
                    stream_frame_info->temp_byte_size, (uint8_t*)stream_frame_info->image_frame, \
                    (uint8_t*)stream_frame_info->temp_frame);
    }

#if defined(_WIN32)
    ReleaseSemaphore(image_sem, 1, NULL);
#elif defined(linux) || defined(unix)
    sem_post(&image_sem);
#endif

    //printf("raw data\n");
    i++;
}

```

display.cpp 中的 display_function，会在等待拿到一帧的数据之后，将图像数据根据原始出图模式，进行不同的转换，然后调用 drm_display 进行显示。

```

while (isRUNNING)
{
    sem_wait(&image_sem);

```

```

        if ((stream_frame_info->frame_output_format == ONLY_IMAGE)||((stream_frame_info->frame_output_format == YUYV_AND_TEMP))
    {
        yuyv2bgr24(stream_frame_info->image_frame, stream_frame_info->width, stream_frame_info->height,rgb_image_frame);
    }
    if(stream_frame_info->frame_output_format == NV12_AND_TEMP)
    {
        nv12_to_yuv444(stream_frame_info->image_frame, stream_frame_info->width*stream_frame_info->height,yuv444_image_frame);
        yuv444_to_rgb(yuv444_image_frame, stream_frame_info->width*stream_frame_info->height,rgb_image_frame);
    }
    drm_display(rgb_image_frame);
    sem_post(&image_done_sem);
}

```

3.发送命令

在 cmd.cpp 的 command_sel 函数中，在输入不同数字后，触发对应的命令。

```

//command thread function
void* cmd_function(void* threadarg)
{
    int cmd = 1;
    while (isRUNNING)
    {
        printf("please select cmd type\n");
        printf("1:get_device_info 2:basic_ffc_update 3:palette_operate 4:temp_measure 5:data_update\n");
        printf("6:set_image_scene_mode 7:image_effection\n");
        scanf("%d", &cmd);
        command_sel(cmd, ((StreamFrameInfo_t*)threadarg));
    }
    printf("cmd thread exit!!\n");
    return NULL;
}

```

4.结束程序

在 sample 里，调用 destroy_pthread_sem 以关闭信号。在 camera 里，调用 ir_video_close 来关闭设备连接。

特殊组件说明

1. 升级功能

1.1 背景

针对 RS300 芯片开发的机芯，英菲提供两种升级方法，第一种是升级包方式，第二种是单文件方式。

升级包方式是将固件、数据统一打包成一个压缩包，由 SDK 进行解压、文件提取和判断需要升级的文件，将差异文件写入机芯。该种方式的优点是封装性较好，用户开发使用简单，但这种方式还未开发完善。缺点在于调整某些算法参数或者伪彩表就需要厂商提供相应的升级包，对厂商依赖度较大，不方便快速调试和开发。

单文件升级方式是用户根据需求，由厂商告知具体升级哪个文件，以及文件存放的机芯目标位置，调用相关的接口。需要厂商提供相应的数据文件以及文件与目标位置的映射表。该种升级方式用户开发起来较为繁琐，且需要对文件的存储结构有一定的了解。优点在于可以快速调试开发。

升级包方式在此不过多介绍，后面文档会完善，下面主要介绍下单文件升级方式。

1.2 机芯的数据区域简介

机芯中的数据可以分为以下五个大的数据区域：Header.bin、default_cfg.bin、system_cfg.bin、user_cfg.bin、secd_cal_cfg.bin。

除 Header.bin 数据区域外，其余四个大的数据区域均由三部分更小的数据区段组成

- *_file_info.bin：大数据区段的**属性文件**，存储着该分区内所有小文件的数据索引信息
- *_json.bin：大数据区段的**配置文件**，存储着机芯的参数配置信息
- *.bin：**零散小文件**

本示例能够操作的数据分为以下两类：

- 普通文件（normal_file）：所有**大数据区域文件**、所有大数据区段内部的**属性文件**和**配置文件**
- 零散文件（scattered_file）：大数据区域内部的所有**零散小文件**

普通文件和零散文件由于所属的文件层级不同，下载功能需要调用示例中不同的接口实现。

1.3 数据更新常见情形

机芯中不同区域的数据稳定性方面存在差异，其中，system_cfg.bin、user_cfg.bin 两个大的数据区域及其内部的各种零散小文件经常需要更新。

当需要更新的文件属于 system_cfg.bin、user_cfg.bin、system_cfg_file_info.bin、system_cfg_json.bin、user_cfg_file_info.bin、user_cfg_json.bin 范畴时，可以参考数据更新流程中的普通文件下载说明进行操作

当需要更新 system_cfg.bin、user_cfg.bin 内部的各种零散小文件时，可以参考数据更新流程中的零散文件下载说明进行操作

下载单文件时，需要按照一定的格式将单文件的目标文件名下发给机芯，以便机芯确定该文件的烧录位置；

所有的烧录文件均有唯一的目标文件名，并可以从目标文件名映射表中查询；可以根据目标文件名中是否带有路径来判断其为普通文件还是零散文件，带有路径的为零

散文件，否则为普通文件。

例如：

isp_cfg0.bin 文件在进行烧录时，其目标文件名为 system_data/isp_cfg0.bin，带有路径，为零散文件

user_cfg.bin 文件在进行烧录时，其目标文件名为其本身 user_cfg.bin，不带路径，为普通文件

1.4 普通文件下载

以下以 user_cfg_json.bin 文件的烧录为例展示普通文件下载功能

运行 SDK 如下图所示，键入 2 再键入 2 再键入 1 触发普通文件下载功能

```
upgrade sample start
-----please select download type-----
1:download upgrade package
2:download single file
2
-----please select download type-----
1:download firmware data
2:write or read single file
2
dev_param = /dev/i2c-1
----- attempt to connect device -----
libiri2c debug [libiri2c.c:198/iri2c_device_open] dev_node = /dev/i2c-1
libircmd debug [libircmd.c:124/ircmd_create_handle] ircmd_create_handle
-----please select download type-----
1:write normal file
2:write scattered file
3:read single file
1
please input target file name
user_cfg_json.bin
please input local file path
/dataset/user_cfg_json.bin
local file length is 2008
libircmd debug [libircmd.c:2928/adv_cfg_file_open] storage_length = 2008
libircmd debug [libircmd.c:2929/adv_cfg_file_open] file_rw_permission = 1
download percentage = 100.00 %
user_cfg_json.bin file data download success!
===== succeed to write normal file =====
```

普通文件下载接口

目标文件名

文件的本地路径

数据更新的进度条

数据下载成功

please input target file name 提示后需要输入待写入文件的目标文件名，该文件名会被下发给机芯，以便机芯确定数据写入的地址，目标文件名可从目标文件名映射表中查询

please input local file path 提示后需要输入数据文件的本地路径（绝对路径或相对于可执行程序的路径）该示例会从该路径指向的文件中读取数据，并将该数据下载至机芯

进入数据下载阶段，SDK 会提示数据更新进度条

当出现 download_success 提示时，表明文件成功写入。

1.5 零散文件下载

以下以 isp_cfg0.bin 文件的烧录为例展示零散文件下载功能，isp_cfg0.bin 文件是组成 system_cfg.bin 大数据区域的零散小文件。

1. 运行 SDK 如下图所示，键入 2 再键入 2 再键入 2 触发零散文件下载功能

```
upgrade sample start
-----please select download type-----
1:download upgrade package
2:download single file
2
-----please select download type-----
1:download firmware data
2:write or read single file
2
dev_param = /dev/i2c-1
----- attempt to connect device -----
libiri2c debug [libiri2c.c:198/iri2c_device_open] dev_node = /dev/i2c-1
libircmd debug [libircmd.c:124/ircmd_create_handle] ircmd_create_handle
-----please select download type-----
1:write normal file
2:write scattered file
3:read single file
2
please input target file name
system_data/isp_cfg0.bin
please input local file path
/dataset/isp_cfg0.bin
local file length is 7137
libircmd debug [libircmd.c:2928/adv_cfg_file_open] storage_length = 7137
libircmd debug [libircmd.c:2929/adv_cfg_file_open] file_rw_permission = 1
download percentage = 57.39 %
download percentage = 100.00 %
libircmd debug [libircmd.c:3120/adv_cfg_file_modify_crc] file_length = 7137
libircmd debug [libircmd.c:3122/adv_cfg_file_modify_crc] crc_value = 0xc04e6643
system_data/isp_cfg0.bin file data download success!
===== succeed to write calibration file =====
```

零散文件下载接口

目标文件名

文件的本地路径

数据更新进度条

数据下载成功

2. please input target file name 提示后需要输入待写入文件的目标文件名，该文件名会被下发给机芯，以便机芯确定数据写入的地址，目标文件名可从目标文件名映射表中查询。

3. please input local file path 提示后需要输入数据文件的本地路径（绝对路径或相对于可执行程序的相对路径）该示例会从该路径指向的文件中读取数据，并将该数据下载至机芯

4. 进入数据下载阶段，SDK 会提示数据更新进度条

5. 当出现 download_success 提示时，表明文件成功写入

1.6 主要的调用接口说明

adv_cfg_file_close_all：升级前首先调用该接口关闭所有文件的读写。（厂商升级协议所需）

test_download_normal_file：可参考该接口进行普通文件的升级，主要调用 config_file_write 接口，输入参数包括目标文件路径、升级类型（普通文件）、文件数据指针、文件大小以及进度条回调函数。

test_download_scattered_file：可参考该接口进行零散文件的升级，主要调用 config_file_write 接口，输入参数包括目标文件路径、升级类型（零散文件）、文件数据指针、文件大小以及进度条回调函数。

test_read_single_file：可参考该接口进行单文件的读取，主要调用 config_file_read 接口，输入参数包括目标文件路径、本地存放路径以及进度条回调函数。

注意：普通文件和零散文件的升级区别在于零散文件需要调用函数接口 adv_cfg_file_modify_crc。

1.7 不同通信接口集成（当前已验证通过 I2C、UART 和 USB 接口）

components/cmd.cpp 文件夹下的 data_update_demo 示例为一个功能较全面的示例，支持通过使用 uart、usb、usb_i2c、i2c 四种命令通路升级数据，只需要通过设置命令通路类型及其设备节点参数即可切换不同的命令通路，但为了方便用户更加快速简单的集成不同命令通路，当前单文件更新方式不需要依赖 pack_tool。下面将简单介绍用户集成不同命令通路的差异点及示例：

I2C 接口更新数据

依赖库：libircam、libircmd、libiri2c、pack_tool

设备节点参数：

```
void* dev_param = NULL;
dev_param = (char*)malloc(20);
memcpy(dev_param, "/dev/i2c-1", 20);
```

UART 接口更新数据

依赖库：libircam、libircmd、libiruart、pack_tool

设备节点参数：

```
void* dev_param = NULL;
dev_param = (char*)malloc(20);
memcpy(dev_param, "/dev/ttyUSB0", 20);
```

USB 接口更新数据

依赖库：libircam、libircmd、libiruvic、libirdfu、pack_tool

设备节点参数：

```
IruvicDevParam_t dev_param = { 0 };
dev_param.pid = 0x0020; //设备的 PID
dev_param.vid = 0x3474; //设备的 VID
dev_param.same_idx = 0; //设备列表的索引
```

USB_I2C 更新数据(Cypress 转接板更新)

由于该通路的上层最终仍是通过 USB 的方式传输数据，故设备节点类型同 USB 通路，更新逻辑与 I2C 接口等同。

依赖库：libircam、libircmd、libiruvic、pack_tool

设备节点参数：

1.8 串口数据升级功能特别说明

机芯对外提供两个串口，一个用于输出调试信息，一个用于发送指令，也用于升级。可参考 search_com 函数搜索出用于升级的串口。如果已知哪个串口用于发送指令，可直接指定串口设备节点。调用 ir_control_open 打开串口，其中 param 即为串口设备节点的字符串。

2.测温二次标定

非制冷红外热像仪对温度敏感，无论模组出厂测温标定如何精确，当用户将模组集成到整机中后，由于热分布的变化、光学结构的变化（加窗口片等），必然引起一定的测温偏差。因此需要在整机端进行二次标定。

3.盲元标定

模组在使用过程中，受到比较严重的机械冲击或静电放电后，有极低的概率出现新增盲元，用户可将新增盲元添加到盲元表中。该功能有多种实现方法：1. 调用固件中的自动标定功能；2. 手动输入新增盲元坐标。

4.测温环境变量修正功能

红外测温的精度受到很多环境参数的影响，例如距离、环境温度、反射温度、湿度、目标发射率等等。Libirtemp SDK 为用户提供了一种测温修正的方法，可以使测温精度更加接近真实值。

具体软件实现：

将机芯读取的当前未经修正的原始温度、用户输入的环境变量参数和环境变量修正表作为参数传入函数 `enhance_distance_temp_correct`(`EnvCorrectParam* env_correct_param`, `uint16_t*correct_table`, `uint32_t table_len`, `float org_temp`, `float* new_temp`)即可得到环境变量修正后的温度。

参数详情介绍如下：

1. 用户输入环境变量修正参数，例如如下设置：

```
EnvCorrectParam env_correct_param = { 0 };
env_correct_param.dist = 0.25; 用户输入修正距离(0~1000m)[double];
env_correct_param.ems = 0.98; 用户输入发射率 ems (0~1)[double];
env_correct_param.hum = 0.45; 用户输入环境湿度(0~1)[double];
env_correct_param.ta = 25; 用户输入大气温度 ta (-273.15~100)[double][°C]
env_correct_param.tu = 25; 用户输入反射温度 tu (-273.15~100)[double][°C];
```

2. 根据机芯当前的增益状态（高/低）打开对应的环境变量修正表（correct table）。后续产品的出货情况可以根据情况从机芯或者是从文件中读取环境变量修正表。环境变量表会随需求的变更及我司的实际测试情况持续更新。此处示例所使用的环境变量修正表为：V265_mini256without_dust_tablets_L。

例如：

```
FILE* fp = fopen("V265_mini256without_dust_tablets_L.bin", "rb");
if (fp == NULL)
{
    printf("fopen V265_mini256without_dust_tablets_L.bin failed\n");
    return NULL;
}
fread(correct_table, 1, sizeof(correct_table), fp);
fclose(fp);
```

3. 第三个参数 `table_len` 是环境变量修正表 (`correct`) 的长度。
4. 第四个参数 `org_temp` (单位 : $^{\circ}\text{C}$) 是机芯读取的当前未经环境变量修正的原始温度 T_0 。
5. 第五个参数 `new_temp` (单位 : $^{\circ}\text{C}$) 是经过环境变量修正后的温度。

软件示例参照 `RS300_SDK_libir_sample` 文件夹中的特殊功能组件 `components` 中的 `temp_measure.cpp` 文件中的 `temp_measure_function` 函数。

5.点线框测温说明

本产品模组内置点线框测温功能，可以分别调用 `libircmd` 库中的接口来获取模组内的特定的点线框及整帧的温度最大/最小值信息。

同时 `libirtemp` SDK 中也提供了点线框的测温功能，使用前提是获取到了一整帧的温度数据，可以分别调用 `get_point_temp`、`get_line_temp`、`get_rect_temp` 函数获取特定的点线框温度信息。为了减小空域噪声的影响，其中点测温的功能是取点周围 $3*3$ 领域内的像素温度值去掉最大最小值后的平均值，线测温 and 框测温均以点测温为基础进行开发。若用户不满意该 SDK 的功能，可自行开发。

6.高精度时钟说明

背景：

为了让客户在设置时间时达到更高的精度，我们提供了高精度时钟方案。

方案：

客端通过硬件 GPIO 中断进行时间同步，需要客端引出 GPIO 与机芯连接。

当下设置时间为 `t_set` 时，先触发 GPIO，此时机芯内部会立刻开始补偿时间计时 `t0`，再下发设置时间指令，此时机芯会获取当前补偿时间计时 `t1`，并设置系统时间为 `t=t1-t0+t_set`。

函数接口：

```
IrLibError_e basic_rtc_current_time_set(IrcmdHandle_t* handle, Rtc_Time_t* rtc_time);
```

使用流程：

该流程在 `libir_sample` 中的 `cmd.cpp` 中实现。

初始化：

```
struct timeval current_time;
struct tm *ptm;
Rtc_Time_t rtc_time;
```

硬件触发 GPIO

```
//这是我在 RV1126 平台测试时引出的引脚，不同的平台根据实际情况调整
system("echo 80 > /sys/class/gpio/export");
system("echo out > /sys/class/gpio/gpio80/direction");
system("echo 1 > /sys/class/gpio/gpio80/value");
```

客端获取时间

```
//获取当前系统时间（根据自身需求设置），如果不需要精确到毫秒或微秒，需初始化为 0，否则会报错
```

```
gettimeofday(&current_time, NULL);
ptm = localtime(&current_time.tv_sec);
rtc_time.day = ptm->tm_mday;
rtc_time.month = ptm->tm_mon + 1;
rtc_time.year = ptm->tm_year + 1900;
rtc_time.hour = ptm->tm_hour;
rtc_time.minute = ptm->tm_min;
rtc_time.second = ptm->tm_sec;
rtc_time.millisecond = current_time.tv_usec / 1000;
rtc_time.microsecond = current_time.tv_usec % 1000;
设置 RTC 时间
ret = basic_rtc_current_time_set(ircmd_handle, &rtc_time);
```

不同模式数据流

在 SDK sample 的 stream_cmd 中，通过 sample.h 文件中修改不同的宏，可以输出不同分辨率和模式的数据流。

宏	传输格式	数据	分辨率	每帧数据大小（字节）
YUYV_AND_TEMP_OUTPUT	YUYV	图像 YUYV+ 信息行+温度 Y16+信息行	640*1026	$640 * (512 + 2) * 2 + 640 * 512 * 2 = 1313280$
YUYV_IMAGE_OUTPUT	YUYV	图像 YUYV	640*512	$640 * 512 * 2 = 655360$
NV12_IMAGE_OUTPUT	NV12	图像 NV12	640*384	$640 * 512 * 1.5 = 491520$
NV12_AND_TEMP_OUTPUT	NV12	图像 NV12+ 信息行+温度 Y16+Dummy	640*900	$640 * 512 * 1.5 + 640 * 2 * 2 + 640 * 512 * 2 + 640 * 2 * 2 = 115200$ 0